

NumPy- Random Number Generation

NumPy provides a comprehensive suite of functions to generate random numbers, which can be very useful for simulations, data analysis, and testing. Here's an overview of some commonly used functions for random number generation in NumPy:

Basic Random Number Generation Function

1) **numpy.random.rand():**-

Generates random numbers from a uniform distribution over [0, 1).

```
import numpy as np
x1=np.random.rand()#Output as Single Value between 0 to 1
x2=np.random.rand(10)#1D Array with 10 elements
x3=np.random.rand(10,10)#2D Array with 100(10row &
10column)elements
print(x1)
print(x2)
print(x3)
```

2) **numpy.random.randn():**-

Generates random numbers from a standard normal distribution (mean = 0, std = 1).

```
import numpy as np
x1=np.random.rand()#Output as Single Value between 0 to 1
x2=np.random.rand(10)#1D Array with 10 elements
x3=np.random.rand(10,10)#2D Array with 100(10row &
10column)elements
print(x1)
print(x2)
print(x3)
```

3) **numpy.random.randint**

Generates random integers between a specified ranges.

```
import numpy as np
x1=np.random.randint(10)#Output single value Between 0 to 10
x2=np.random.randint(1,10)#Output single value Between 1 to 10
x3=np.random.randint(1,10,20)#Output 20 elements Between 1 to 10
x4=np.random.randint(100,size=(4,5))#Output 2D Array between 1 to 100
print(x1)
print(x2)
print(x3)
print(x4)
```

4) **numpy.random.choice():**-

The choice() method allows you to generate a random value based on an array of values.

The choice() method takes an array as a parameter and randomly returns one of the values.

```
import numpy as np
x1=np.random.choice(range(1,100))
x2=np.random.choice([1,2,3,4,5,6,7,8])
x3=np.random.choice([1,2,3,4,5,6,7,8],size=5)
x4=np.random.choice([1,2,3,4,5,6,7,8],size=(5,4))
print(x1)
print(x2)
print(x3)
print(x4)
```

5) **numpy.random.shuffle():-**

Shuffles the contents of an array in-place.

```
import numpy as np
array = [1, 2, 3, 4, 5]
data=np.random.shuffle(array) # Shuffles the array in-place
print(data)#None output should be display
print(array)
```

6) **numpy.random.permutation():-**

Randomly permutes a sequence or returns a permuted range.

```
import numpy as np
array = [1, 2, 3, 4, 5]
data=np.random.permutation(array) # Shuffles the array in-place
print(data)#print after shuffle
print(array)#same as array
```

Setting with Seed Function

The numpy.random.seed() function in NumPy is used to set the seed for the random number generator. This function initializes the random number generator with a specific seed value, which ensures that you get the same sequence of random numbers every time you run the code with that seed. This is important for reproducibility, consistency in testing, and debugging.

Why Use numpy.random.seed()?

✓ **Reproducibility:**

Using the same seed value ensures that you obtain the same sequence of random numbers every time. This is crucial for scientific experiments, simulations, and data analysis where you need to replicate results.

✓ **Testing and Debugging:**

Having predictable outputs is important for finding bugs and testing code. By setting a seed, you can generate the same random sequences, making it easier to test and debug your code.

```
import numpy as np
# Set the seed
np.random.seed(42)
# Generate random numbers
random_numbers = np.random.rand(5)#print each time same value due to
seed function
print(random_numbers)
```

Random Number Generators for Various Distributions

In NumPy, a distribution refers to a probability distribution that describes how values are spread or dispersed. NumPy provides functions to generate random numbers according to various probability distributions, each of which has its own set of parameters that define its shape and properties. These distributions are useful for simulations, statistical analyses, and various applications in data science and machine learning.

1) Uniform Distribution:

- ✓ Generates numbers evenly distributed within a specified range.
- ✓ Function: `numpy.random.uniform(low=0.0, high=1.0, size=None)`
 - low: Range ka lower bound
 - high: Range ka upper bound
 - size: Generate hone wale random numbers ka size ya shape

```
import numpy as np
ud1 = np.random.uniform(low=0.0, high=10.0, size=10)
ud2= np.random.uniform(low=0.0, high=10.0, size=(10,10))
print(ud1)
print(ud2)
```

2) Normal Distribution(Gaussian Distribution):

- ✓ Generates numbers with a bell-shaped curve, centered around a mean (loc) with a specified standard deviation (scale).
- ✓ Function: `numpy.random.normal(loc=0.0, scale=1.0, size=None)`
 - loc (mean): Distribution ka center point ya average value.
 - scale (standard deviation): Values ka spread ya dispersion around the mean.
 - size: Generate hone wale random numbers ka size ya shape.

```
import numpy as np
nd1 = np.random.normal(loc=0.0, scale=1.0, size=10)
```

```
nd2 = np.random.normal(loc=0.0, scale=1.0, size=(5,10))
print(nd1)
print(nd2)
```

3) Binomial Distribution:

- ✓ Generates numbers representing the number of successes in a sequence of independent trials.

- ✓ Function: `numpy.random.binomial(n, p, size=None)`

n: Number of trials (yaani total number of experiments).

p: Probability of success in each trial.

size: Number of random numbers to generate.

```
import numpy as np
```

```
bd1 = np.random.binomial(n=10, p=0.5, size=10)
```

```
bd2 = np.random.binomial(n=10, p=0.5, size=(5,4))
```

```
print(bd1)
```

```
print(bd2)
```

4) Poisson Distribution:

- ✓ Generates numbers representing the number of events occurring in a fixed interval of time or space.

- ✓ Function: `numpy.random.poisson(lam=1.0, size=None)`

lam (lambda): Expected number of occurrences (mean rate) in a fixed interval.

size: Number of random numbers to generate.

```
import numpy as np
```

```
pd1 = np.random.poisson(lam=5, size=10)
```

```
pd2 = np.random.poisson(lam=5, size=(5,5))
```

```
print(pd1)
```

```
print(pd2)
```

Assignment

- 1) Write a NumPy program to generate five random numbers from the normal distribution.
- 2) Write a NumPy program to generate six random integers between 10 and 30.
- 3) Write a NumPy program to create a 3x3x3 array with random values.
- 4) Write a NumPy program to create a 5x5 array with random values and find the minimum and maximum values.
- 5) Write a NumPy program to create a random 10x4 array and extract the first five rows of the array and store them into a variable.
- 6) Write a NumPy program to shuffle numbers between 0 and 10 (inclusive).
- 7) Write a NumPy program to create a random vector of size 10 and sort it
- 8) Write a NumPy program to check two random arrays are equal or not.
- 9) Write a NumPy program to create random vector of size 15 and replace the maximum value by -1.
- 10) Write a NumPy program to find the most frequent value in an array.